

Afterlife — Design Spec (v1)

3 October 2026 · Status: approved; growth rule revised 3 Oct (see §11) · Owner: Ujjwal (director) · Builder: Claude

1. Purpose and success criteria

Afterlife is a calm isometric web game in which the player grows plants over abandoned urban ruins until nature has taken each scene back. It's a personal project with two goals: **fun to make and play**, and **a portfolio piece** that shows a polished, playable game, a landing page and clean public code.

The main inspiration is *Cloud Gardens* (Noio, 2021); see docs/research/cloud-gardens-research.pdf. Afterlife borrows that game's core mechanics. It uses **no names, art, levels or audio** from it.

v1 is done when all of these are true:

1. All 5 levels can be completed. An automated test proves each level's reference solution reaches its target.
2. The game runs smoothly on current desktop Chrome, Safari and Firefox and on mobile Safari and Chrome, with mouse and touch.
3. The game and its landing page are live on Vercel.
4. The code is in a **public** GitHub repo with a README, an MIT licence for the code and credits for the CC0 art.
5. Ujjwal has played all 5 levels and is happy with the feel.

2. Decisions already made

Topic	Decision
Name	Afterlife
Genre	Calm puzzle, Cloud Gardens-style; no timer, no fail state
View	Isometric 2.5D on a tile grid, with the camera rotating in 90° steps
Roles	Claude writes all code; Ujjwal directs and decides design and feel
Stack	Phaser + TypeScript + Vite, tests with Vitest, hosted on Vercel

Topic	Decision
Art	Hybrid: Kenney.nl CC0 isometric tiles and props, recoloured to the Afterlife palette, plus code-drawn plants and effects
Audio	Generative ambient music (Tone.js) plus CC0 sound effects
Scope	5 levels, title screen, level select, landing page
Repo	Public GitHub

3. Gameplay rules

These rules are written precisely because the growth engine is built and tested from them.

3.1 The grid

A level is a rectangular grid (8×8 by default; each level sets its own size). Each **tile** has three layers:

Layer	Values
Ground	soil, concrete (both plantable and counted by the meter) or blocked (walls, holes, edges: nothing goes there and the meter ignores it)
Object	Empty, a ruin (fixed scenery placed by the level) or scrap (placed by the player). At most one object per tile
Plant	Empty or one plant cell. At most one plant cell per tile

Distance between tiles is **Manhattan distance** ($|dx| + |dy|$), so a radius shows as a diamond, which reads naturally in isometric view.

3.2 Items

Scrap (placed by the player, comes in batches):

Size	Examples	Growth radius
Small	tyre, can, traffic cone	1
Medium	crate, barrel, road sign	2
Large	car shell	3

Ruins (placed by the level) are pre-existing objects such as a bench or a bus shelter. Each ruin has a size (small, medium or large), which decides which plants can grow on it. Ruins have **no growth radius**, but they count toward the meter and must be covered, just like scrap.

Plants (from seeds):

Plant	Can grow on	Max stage	When grown (stage = max) and fed again
Moss	Ground, small scrap, small ruins	2	Spreads into every eligible neighbouring tile
Vine	Ground and any scrap or ruin	3	Climbs onto every eligible neighbour holding an object; with none, creeps onto 1 random bare neighbour
Flower	Ground only	3	Grows a bloom (max 1 at a time) that the player can harvest
Bamboo	Ground only	5	Does not spread; it's a tall, narrow cover with a large visual height

“Neighbour” means the 4 orthogonally adjacent tiles. A tile is eligible for a plant when its ground isn't blocked, it has no plant cell, and its object (if any) is one that plant can grow on. A spread cell starts at stage 1, belongs to the same plant, and can be fed and spread in turn.

3.3 Turns: what happens when scrap is placed

The game has no clock. It advances **only when the player places scrap**:

1. The scrap occupies its tile.
2. Every plant cell within the scrap's radius receives **one growth tick**:
 - Below max stage: stage + 1.
 - At max stage: do the plant's “when grown” action from the table above.
3. A cell that is at max stage and has no eligible neighbour shows as **“blocked”**.
4. The meter updates.

Placing a seed does **not** cause any growth.

Randomness, such as which neighbour moss spreads into, uses a **seeded random number generator**. The same level with the same moves always produces the same garden, which makes undo and testing reliable.

3.4 The player's inventory

- **Seeds:** a count per plant type, with a starting amount set by the level.
- **Scrap batches:** an ordered list of batches. The tray shows the current batch, and dots show how many batches remain. The next batch arrives when the current one is used up.
- **Harvesting:** tapping a bloom removes it and gives **+1 seed**. The level sets which plant type the seed is, defaulting to the flower's own type.

3.5 Placement rules

- A **seed** can go on any tile where that plant is eligible (§3.2).
- **Scrap** can go on any non-blocked tile that has **no object and no plant**.
- Invalid tiles show a soft “no” highlight while you hover over them; nothing happens on click.

3.6 The meter and winning

- A tile counts as **covered** when it has a plant cell at **stage 1 or higher**. A fresh seed (stage 0) does not count.
- So an **object** (scrap or ruin) counts as covered when its tile has a grown-in plant cell.
- **Coverage** = (covered non-blocked tiles) ÷ (all non-blocked tiles). Every object sits on a non-blocked tile, so it's included automatically.
- Each level sets a **target**, for example 80%. The meter on screen fills toward that target, so a full meter means the target has been reached.

Placing scrap on a bare tile uses that tile until something covers it. That's the nature-versus-scrap balance carried over from Cloud Gardens.

Win: when coverage reaches the target, the “scene restored” moment plays: colour washes in, the music swells and the camera makes one slow turn. Then the player chooses **Next** or **Keep decorating**.

Stuck: if all scrap is used up (the tray and every batch are empty) and the target hasn't been reached, nothing can grow any more, so a calm panel appears saying “The garden rests...” with **Undo** and **Restart**. It isn't a game-over screen.

3.7 Undo and restart

- **Undo** reverses the last action (place seed, place scrap or harvest). It's unlimited within a level and works from snapshots of the whole level state.
- **Restart** resets the level to its starting state.

3.8 Making the rules visible

This fixes the biggest complaint about Cloud Gardens:

- **Hovering** with scrap selected shows its **radius diamond**, and every plant cell inside it gently glows.
- Plant cells show a state: **growing**, **grown** or **blocked**. Each state has its own icon, not only colour, so colour-blind players can tell them apart.

- A **one-line hint** introduces each level's new idea.

4. The five levels

#	Scene	Introduces	Teaching goal
1	Bus Stop (shelter and bench)	Moss, tyre (small scrap)	Scrap makes nearby plants grow
2	Rooftop (water tanks, AC unit)	Vine, crate (medium scrap)	Covering objects counts; vines climb
3	Petrol Station (pumps, car)	Flower and harvesting	Harvest blooms to keep your seed supply
4	Railway Platform (long and narrow, 4×12)	Bamboo	Planning in tight spaces
5	Playground (slide, swings)	Car shell (large scrap)	Finale: combine everything

Each level is a **data file** (`src/levels/NN-name.json`) containing the grid size, ground layout, ruins, starting seeds, scrap batches, target, random seed, hint text and a **reference solution**, the list of moves the solvability test replays. Exact numbers are tuned during playtesting.

5. Screens and flow

1. **Title screen:** the “Afterlife” logo over a slowly overgrowing scene, with Play, Settings (volume, mute, reduced motion) and Credits.
2. **Level select:** 5 diorama cards. Each card is grey until completed, then green. Levels unlock in order.
3. **Play screen:** the diorama in the centre, the meter at the top, the item tray at the bottom (seeds and the current scrap batch), and buttons for Undo, Restart, Rotate (left and right), Mute and Menu.
4. **Scene restored:** the overlay described in §3.6.

Controls: - **Mouse:** click a tray item, then hover to see the preview and click to place. Right-click or Esc deselects. Use the wheel to zoom and Q/E to rotate. - **Touch:** tap a tray item, then tap a tile to see the preview and tap the same tile again to place. Pinch to zoom.

6. Art and audio direction

- **Palette:** dusty greys, rust and faded concrete at the start, with greens and soft flower colours as the scene recovers. Light is warm and slightly hazy.

- **Ruins and scrap:** Kenney CC0 isometric sprites, recoloured at build time or with Phaser tinting to fit the palette. Specific packs are chosen during setup, and their licence files are kept in `assets/LICENSES/`.
- **Plants:** drawn in code with Phaser Graphics and cached as textures: stems, leaves, petals and moss clumps, with small seeded variation so no two gardens look the same. Growth steps animate with a short tween.
- **Audio:** an ambient bed generated with Tone.js (soft pads and wind) that gets fuller as coverage rises, plus CC0 sound effects for placing, harvesting and completing a level. Sound starts after the first click or tap. The mute and volume settings are saved.
- **Reduced motion:** when the system setting `prefers-reduced-motion` is on, or the in-game setting is enabled, camera turns and large tweens are replaced by fades.

7. Technical architecture

7.1 Modules

Module	Folder	Responsibility	Depends on
Engine	<code>src/engine/</code>	Pure TypeScript game rules: grid, plants, growth ticks, placement checks, meter, inventory, undo, seeded random numbers. It never imports Phaser.	none
Levels	<code>src/levels/</code>	Level data files and a loader that validates them	Engine types
Iso renderer	<code>src/render/iso/</code>	Converts grid coordinates to the screen and back for 4 rotations; draws ground, objects and depth order	Phaser, Engine state (read-only)
Plant renderer	<code>src/render/plants/</code>	Draws and animates plant cells from Engine state	Phaser, Engine state
Scenes	<code>src/scenes/</code>	Boot (loading), Title, LevelSelect, Play; Play connects input → Engine → renderers	All of the above

Module	Folder	Responsibility	Depends on
UI	src/ui/	Tray, meter, buttons, overlays	Phaser
Audio	src/audio/	Ambient music and sound effects	Tone.js
Save	src/save/	Reads and writes progress and settings in localStorage under the key <code>afterlife.save.v1</code> . Every access is wrapped in try/catch, and the game still works if storage is unavailable	none
Landing	landing/	Static landing page: hero animation, pitch, screenshots, Play button	none (plain HTML/CSS/TS)

Data flow: a player input reaches the Play scene, which calls an Engine action such as `placeScrap(x, y)`. The Engine returns a new state plus a list of **events** (`grew`, `spread`, `bloomed`, `blocked`, `won`). The renderers and audio animate from those events. The Engine never calls rendering code.

7.2 Build and hosting

- Vite builds one site in two parts: `/` (landing page) and `/play/` (the game).
- It's deployed to Vercel from the public GitHub repo, and each push creates a preview link.
- **Performance budget:** a steady 60 fps on a mid-range laptop and smooth play on a recent phone. The first load should be under 3 MB compressed.

7.3 Error handling

- **Invalid level file:** the loader rejects it with a clear developer error during build or test, never silently in the live game.
- **Storage unavailable** (private mode or blocked): the game plays normally, and progress is simply not saved.
- **Audio blocked or failed:** the game plays silently, and the mute icon shows the off state.
- **Unexpected runtime error:** a soft panel says “Something went wrong — Restart level” instead of a frozen screen, and the details go to the console.

8. Testing

What	How
Engine rules	Vitest unit tests for every rule in §3: placement checks, growth ticks, spreading, vine preference, blooms and harvesting, the meter, undo, and repeatable seeded random numbers
Level solvability	For each level, replay its reference solution in the Engine and check that it reaches the target
Level files	Every level file passes the loader's validation
Real play	Claude plays each level in Chrome on desktop and in a mobile-sized window and checks visuals, input, audio and the win flow
Feel	Ujjwal playtests; tuning changes go into the level data files

9. Out of scope for v1

Sandbox mode, photo mode, video export, story or dialogue, accounts and cloud saves, analytics, monetisation, itch.io release, extra languages, and more than 5 levels. Any of these can be a later version once v1 has shipped.

10. Repository layout

Afterlife/

docs/	research, specs, plans
landing/	landing page
src/	
engine/	levels/ render/iso/ render/plants/ scenes/ ui/ audio/ save/
assets/	Kenney sprites (recoloured), sound effects, LICENSES/
tests/	Vitest tests (engine + level solvability)
README.md	what it is, how to play, how to run, credits
LICENSE	MIT (code); art stays under its own CC0 licence

11. Change log

- **3 Oct 2026 — growth rule revised (approved by Ujjwal).** In the first version, moss and vines spread one tile per growth tick. The level solver showed that rule caps every level at about 17–25% coverage, because scrap fills the scene faster than plants can cover it. Grown moss now spreads into every eligible neighbour, and vines climb onto every neighbouring object (or creep one tile on bare ground). Each level also gets about twice as many seeds and scrap.